

Class Information

- **Lecturer**

- Prof. Lynn Choi, School of Electrical Eng.
- Phone: 3290-3249, Engineering Building 411, lchoi@korea.ac.kr,
- TA: 이도훈, 3290-3896, luke0911@korea.ac.kr

- **Time**

- Thu 10:30am – 12:15pm

- **Place**

- New Engineering Building 110

- **Textbook**

- “*C로 쓴 자료구조론*”, Horowitz, Shani, Anderson-Freed, 이석호 역, 교보문고, 2nd Edition, 2008.

- **References**

- “*Fundamentals of Data Structures in C*”, Horowitz, Shani, Anderson-Freed
Horowitz, Shani, Anderson-Freed, Silicon Press, 2nd Edition, 2008.

- **Class homepage**

- <http://kulms.korea.ac.kr>, <http://it.korea.ac.kr> : slides, announcements



데이터 구조

1장. 기본 개념



고려대학교
전기전자전파공학부
최 린



高麗大學校

Computer System Laboratory

시스템 생명 주기 (1/2)

- 요구사항(requirements)
 - 프로젝트의 목적을 정의한 명세들의 집합
 - 입력과 출력에 관한 정보를 기술
- 분석(analysis)
 - 문제들을 다룰 수 있는 작은 단위들로 나눔
 - 상향식(bottom-up)/하향식(top-down) 접근 방법
 - 상향식 – 코딩 중심, 프로젝트의 기본 계획이 없으므로 오류 빈번
 - 하향식 – 프로그램을 다룰 수 있는 작은 단위로 나눔
- 설계(design)
 - 프로그램에 필요한 자료 객체
 - 추상 데이터 타입(abstract data type) 생성
 - 자료에 수행될 연산 알고리즘의 명세



시스템 생명 주기(2/2)

- 정제와 코딩(refinement and coding)
 - 데이터 객체에 대한 표현 선택
 - 자료 객체의 표현 방법이 종종 연관된 알고리즘의 효율성을 결정
 - 수행되는 연산에 대한 알고리즘 작성
 - 특정 프로그래밍 언어를 사용한 구현
 - 구문 오류 (syntax error) 제거
- 검증(verification)
 - 정확성 증명(correctness proof)
 - 때때로 수학적 기법들을 이용해서 증명
 - 테스트(testing)
 - 다양한 입력 데이터를 이용한 프로그램의 테스트 (기능 검사)
 - 프로그램의 성능 검사
 - 오류 제거(error removal)
 - 독립적 단위로 테스트 한 후 전체 시스템으로 통합



포인터와 메모리 할당(1/3)

- 포인터

- C언어에서는 어떤 타입 T에 대해 T의 포인터 타입이 존재
- 포인터 타입의 실제 값은 메모리 주소가 됨 (*음이 아닌 정수*)
- 포인터 타입에 사용되는 연산자

- & : 주소 연산자
- * : 역참조 연산자

Ex) i(정수 변수), pi(정수에 대한 포인터)

```
int i, *pi
```

```
pi = &i;
```

i에 10을 저장하기 위해서는 다음과 같이 할 수 있다.

```
i = 10; 또는 *pi = 10;
```

- 널포인터: 어떤 객체도 가리키지 않는 특수한 포인터
 - 정수 0값으로 표현
 - 널포인터에 대한 검사

```
int (pi == Null) 또는 if(!pi)
```



포인터와 메모리 할당(2/3)

- 동적 메모리 할당
 - 프로그램을 작성할 당시 얼마나 많은 공간이 필요한지 알 수 없을 때 사용
 - 힙(heap) 기법
 - 새로운 메모리 공간이 필요할 때마다 함수 malloc을 호출해서 필요한 양의 공간을 요구

```
int i, *pi;  
float f, *pf;  
pi = (int *) malloc(sizeof(int));  
pf = (float *) malloc(sizeof(float));  
*pi = 1024;  
*pf = 3.14;  
printf("an integer = %d, a float = %f\n", *pi, *pf);  
free(pi);  
free(pf);
```

메모리 할당과 반환



포인터와 메모리 할당(3/3)

- 포인터의 위험성

- 포인터가 대상을 가리키고 있지 않을 때는 값을 전부

null로 정하는 것이 바람직

- 이는 프로그램 범위 밖이나 합당하지 않은 메모리 영역을 참조할 가능성이 적어짐

- 명시적인 타입 변환(type cast)

```
pi = malloc(sizeof(int));    /* pi에 정수에 대한 포인터를 저장 */  
pf = (float *) pi;          /* 정수에 대한 포인터를 부동소수에 대한  
                             포인터로 변환 */
```



알고리즘 명세(1/9)

- 알고리즘(Algorithm)
 - 특정한 일을 수행하기 위한 명령어들의 유한 집합
 - 조건
 - 입력 : (외부) 데이터 ≥ 0
 - 출력 : 결과 ≥ 1
 - 명백성(definiteness) : 각 명령들은 명확하고 모호하지 않아야 한다
 - 유한성(finiteness) : 한정된 시간 안에 반드시 종료
 - 유효성(effectiveness) : 각 명령은 종기와 연필로 수행될 수 있게 기본적으로 해야 하고 실행 가능하여야 한다
 - 프로그램과 알고리즘
 - $\text{program} \equiv \text{algorithm}$ (때로 운영체제와 같은 프로그램은 종료하지 않으므로 알고리즘과 프로그램을 구별하기도 함)
 - 알고리즘의 기술
 - 자연어
 - 흐름도(flowchart): 알고리즘이 작고 단순한 경우
 - 이 책에서는 C와 자연어를 혼용



알고리즘 명세(2/9)

- 예제[선택 정렬(selection sort)] : $n \geq 1$ 개의 서로 다른 정수를 정렬
 - 정렬되지 않은 정수들 중에서 가장 작은 값을 찾아서 정렬된 리스트 다음 자리에 놓음
 - 정수들이 배열(array)에 저장

```
for ( i = 0; i < n; i++) {  
    list[i]에서부터 list[n-1]까지의 정수 값을 검사한 결과  
    list[min]이 가장 작은 정수 값이라 하자;  
    list[i]와 list[min]을 서로 교환;  
}
```

< 선택 정렬 알고리즘 >

- 최소 정수를 찾는 작업
 - 최소 정수가 list[i]라 가정
 - List[i]와 list[i+1]~list[n-1] 비교
 - 더 작은 정수를 만나면 새로운 최소정수로 선택



알고리즘 명세(3/9)

- 최소 정수 값을 list[i] 값과 교환하는 작업
 - 함수 정의 : swap(&a, &b)

```
Void swap(int *x, int *y)
/* 매개 변수 x, y는 정수형을 갖는 포인터 변수이다. */
{
    int temp = *x; /* temp변수를 int로 선언하고 x가 가르키는 주소의
                     내용을 지정한다. */
    *x = *y /* y가 가르키는 주소의 내용을 x가 가르키는 주소에
             저장한다. */
    *y = temp /* temp의 내용을 y가 가르키는 주소에 저장한다. */
}
```

< swap 함수 >

- 매크로 정의: 어떤 데이터 타입에도 사용 가능
#define SWAP(x,y,t) ((t) = (x), (x) = (y), (y) = (t))



알고리즘 명세 (4/9)

```
#include <stdio.h>
#include <math.h>
#define MAX_SIZE 101
#define SWAP(x, y, t) ((t) = (x), (x) = (y), (y) = (t))
void sort(int [], int);      /* selection sort */
void main(void)
{
    int i, n;
    int list[MAX_SIZE];
    printf("Enter the number of numbers to generate: ");
    scanf("%d", &n);
    if(n<1 || n> MAX_SIZE) {
        fprintf(stderr, "Improper value of n\n");
        exit(EXIT_FAILURE);
    }
    for (i=0; i<n; i++) { /* randomly generate numbers*/
        list[i] = rand()%1000;
        printf("%d", list[i]);
    }
}
```



알고리즘 명세 (5/9)

```
sort(list, n);
printf("\n Sorted array:\n");
for(i=0; i<n; i++)  /*print out sorted numbers */
    printf("%d", list[i]);
printf("\n");
}
void sort(int list[], int n)
{
    int i, j, min, temp;
    for(i=0; i<n-1; i++) {
        min = i;
        for (j = i+1; j<n; j++)
            if(list[j] <list[min])
                min =j;
        SWAP(list[i], list[min], temp);
    }
}
```



알고리즘 명세 (6/9)

- 정리 1.1

- 함수 $\text{Sort}(\text{list}, n)$ 는 $n \geq 1$ 개의 정수를 정확하게 정렬한다.
그 결과는 $\text{list}[0], \dots, \text{list}[n-1]$ 로 되고 여기서 $\text{list}[0] \leq \text{list}[1] \leq \text{list}[n-1]$ 이다.

- 증명

- $i=q$ 에 대해, 외부 for 루프가 완료되면 $\text{list}[q] \leq \text{list}[r]$,
 $q < r < n$ 이다.

다음 반복에서는 $i > q$ 이고 $\text{list}[0]$ 에서 $\text{list}[q]$ 까지는 변하지 않는다.

따라서 바깥 for 루프를 마지막으로 수행하면(즉, $i=n-2$),
 $\text{list}[0] \leq \text{list}[1] \leq \dots \leq \text{list}[n-1]$ 가 된다.



알고리즘 명세 (7/9)

- 예제[이진탐색(binary search)]: 정수 searchnum이 배열 list에 있는지 검사
 - $List[0] \leq list[1] \leq \dots \leq list[n-1]$
 - 주어진 정수 searchnum에 대해 $list[i]=searchnum$ 인 인덱스 i 를 반환
 - 없는 경우는 -1 반환
 - 초기값 : $left=0, right=n-1$
 - List의 중간 위치 : $middle=(left+right)/2$
 - $List[middle]$ 와 searchnum을 비교
 - 1) $searchnum < list[middle]$:
 $list[0] \leq searchnum \leq list[middle-1]$
 $right = middle-1$
 - 2) $searchnum = list[middle]$: middle을 반환
 - 3) $searchnum > list[middle]$:
 $list[middle+1] \leq searchnum \leq list[n-1]$
 $left = middle+1$



알고리즘 명세 (8/9)

```
while (there are more integers to check) {  
    middle = (left + right) / 2;  
    if(searchnum < list[middle])  
        right = middle-1;  
    else if (searchnum == list[middle])  
        return middle;  
    else if = middle +1;  
}
```

< 정렬된 리스트 탐색 >

```
int compare(int x, int y)  
{ /* compare x and y, return -1 for less than, 0 for equal, 1 for greater */  
    if (x<y) return -1;  
    else if (x == y) return 0;  
    else return 1;  
}
```

< 두 정수의 비교 >



알고리즘 명세 (9/9)

- 비교 연산 : 함수, 매크로

- 첫 번째 수치 값이 두 번째 수치의 값보다 작을 때 음수 반환
- 두 수치 값이 같은 경우 0을 반환
- 첫 번째 수치 값이 두 번째 수치 값보다 큰 경우 양수 반환

```
#define COMPARE(x, y) ((x)<(y) ? -1 : (x) == (y) ? 0 : 1)
```

```
int binsearch(int list[], int searchnum, int left, int right)
{
    /* search list [0] <= list[1] <= ... <= list[n-1] for searchnum.
       Return its position if found. Otherwise return -1 */
    int middle;
    while (left <= right) {
        middle = (left + right) / 2;
        switch (COMPARE(list[middle], searchnum)) {
            case -1: left = middle + 1;
                    break;
            case 0 : return middle;
            case 1 : right = middle - 1;
        }
    }
    return -1;
}
```



순환 알고리즘(1/4)

- Recursive function (재귀 함수 / 순환 함수)
 - A function is defined in terms of itself
 - 수행이 완료되기 전에 자기 자신을 다시 호출
 - 직접 순환 : direct recursion - A calls A
 - 간접 순환 : indirect recursion - A calls B calls C calls A
- 예제
 - Fibonacci 수: $f(n) = f(n-1) + f(n-2)$ where $f(0) = 0$ and $f(1) = 1$
 - Factorial: $fact(n) = n * fact(n-1)$ where $fact(1) = 1$
 - 이항 계수 (binomial coefficient)

$$\binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1}$$



순환 알고리즘(2/4)

- 예제: binary search (이진 탐색)

```
int binsearch(int list[], int searchnum, int left, int right)
{ /* search list [0] <= list[1] <= ... <= list[n-1] for searchnum.
   Return its position if found. Otherwise return -1 */
  int middle;
  if (left <= right) {
    middle = (left + right) / 2;
    switch (COMPARE(list[middle], searchnum)) {
      case -1: return
                binsearch(list, searchnum, middle+1, right);
      case 0 : return middle;
      case 1 : return
                binsearch(list, searchnum, left, middle-1);
    }
  }
  return -1;
}
```

< 이진탐색에 대한 순환 구현 >



순환 알고리즘(3/4)

- 예제: permutation (순열)
 - $n \geq 1$ 개의 원소를 가진 집합에서의 모든 가능한 순열
 - $\{a, b, c\} : \{(a,b,c), (a,c,b), (b,a,c), (b,c,a), (c,a,b), (c,b,a)\}$
 - N원소 : $n!$ 개의 상이한 순열
 - $\{a, b, c, d\}$
 - 1) a로 시작하는 $\{b,c,d\}$ 의 모든 순열
 - 2) b로 시작하는 $\{a,c,d\}$ 의 모든 순열
 - 3) c로 시작하는 $\{a,b,d\}$ 의 모든 순열
 - 4) d로 시작하는 $\{a,b,c\}$ 의 모든 순열



순환 알고리즘(4/4)

- 초기 함수 호출 : $perm(list, 0, n-1)$

```
void perm(char *list, int i, int n)
{
    /* generate all the permutations of list[i] to list[n] */
    int j, temp;
    if(i == n) {
        for(j=0; j<=n; j++)
            printf("%c", list[j]);
        printf("\n");
    }
    else {
        /* list[i] to list[n] has more than one permutation,
        generate these recursively */
        for(j= i; j<=n; j++){
            SWAP(list[i], list[j], temp);
            perm(list, i+1, n);
            SWAP(list[i], list[j], temp);
        }
    }
}
```



데이터 추상화(1/4)

- C언어의 기본 자료형 : char, int, float, double
- 자료의 그룹화 : 배열(array), 구조(structure)
 - int list[5] : 정수형 배열
 - 구조

```
struct student {  
    char lastName;  
    int studentId;  
    char grade;  
}
```

- 포인터 자료형 : 정수형, 실수형, 문자형, float형 포인터
 - int i, *pi;
- 사용자 정의(user-defined) 자료형



데이터 추상화(2/4)

- 정의 : 자료형 (data type)
 - 객체(object)와 그 객체 위에 작동하는 연산(operation)들의 집합
 - int : {0, +1, -1, +2, -2, ..., INT_MAX, INT_MIN}
 - 정수 연산 : +, -, *, /, %, 테스트 연산자(==, !=), ...
 - ++와 같은 전위(prefix) 연산자
 - +와 같은 중위(infix) 연산자
 - 연산: 이름, 매개 변수, 결과가 명세
- 자료형의 객체 표현
 - char형: 1바이트 비트 열
 - int형: 2 또는 4바이트
 - 구체적인 내용을 사용자가 모르도록 하는 것이 좋은 방법
 - ⇒ 객체 구현 내용에 대한 사용자 프로그램의 독립성



데이터 추상화(3/4)

- 정의 : 추상 자료형(ADT: abstract data type)
 - 객체의 명세와 그 연산의 명세가 그 객체의 표현과 연산의 구현으로부터 분리된 자료형
 - 명세와 구현을 명시적으로 구분
 - Ada – package, C++ – Class
- ADT 연산의 명세
 - 함수 이름, 매개 변수형, 결과형, 함수가 수행하는 기능에 대한 기술
 - 내부적 표현이나 구현에 대한 자세한 설명은 필요 없음
- 자료형의 함수는 다음과 같이 분류
 - 생성자(creator)/구성자(constructor) : 새 인스턴스를 생성
 - 변환자(transformer) : 한 개 이상의 서로 다른 인스턴스를 이용하여 지정된 형의 인스턴스를 만듦
 - 관찰자(observers)/보고자(reporter) : 자료형의 인스턴스에 대한 정보를 제공



데이터 추상화(4/4)

- 예제[추상 데이터 타입 NaturalNumber]
 - 객체(objects) : 0에서 시작해서 컴퓨터상의 최대 정수 값 (INT_MAX)까지 순서화된 정수의 부분 범위이다.
 - 함수(functions) : for all $x, y \in \text{Nat_Number}$, TRUE, FALSE $\in \text{Boolean}$ 에 대해, 여기서 +, -, <, 그리고 ==는 일반적인 정수 연산자이다.

```
NaturalNumber Zero()      ::= 0
Boolean IsZero(x)         ::= if(x) return FALSE
                           else return TRUE
Boolean Equal(x, y)       ::= if(x==y) return TRUE
                           else return FALSE
NaturalNumber Successor   ::= if(x == INT_MAX) return x
                           else return x+1
NaturalNumber Add(x,y)    ::= if((x+y)<= INT_MAX) return x+y
                           else return INT_MAX
NaturalNumber Subtract(x,y) ::= if(x<y) return 0
                           else return x-y
End NaturalNumber
```



성능 분석

- 성능 평가(performance evaluation)
 - 성능분석(performance analysis)
 - 시간과 공간의 추산
 - 복잡도 이론(complexity theory)
 - 성능측정(performance measurement)
 - 컴퓨터 의존적 실행 시간
- 정의
 - 공간 복잡도(space complexity)
 - 프로그램을 실행시켜 완료하는데 필요한 (메모리) 공간의 양
 - 시간 복잡도(time complexity)
 - 프로그램을 실행시켜 완료하는데 필요한 (컴퓨터) 시간의 양



공간 복잡도(1/3)

- 프로그램에 필요한 공간
 - 고정 공간 요구
 - 프로그램 입출력의 횟수나 크기와 관계없는 공간 요구
 - 명령어 공간, 단순 변수, 고정 크기의 구조화 변수, 상수
 - 가변 공간 요구
 - 문제의 특정 인스턴스, I , 에 의존하는 공간
 - 가변 공간, 스택 공간
- 공간 요구량 : $S(P) = c + S_p(I)$
- 예제 : abc
 - 고정 공간 요구만을 가짐 $\Rightarrow S_{abc}(I) = 0$

```
float abc(float a, float b, float c)
{
    return a+b+b*c + (a+b-c) / (a+b) + 4.00;
}
```



공간 복잡도(2/3)

- [예제] sum
 - 가변 공간 요구: 배열(크기 n)
 - 함수에 대한 배열의 전달 방식에 따라 다르다
 - Pascal : 값 호출(call by value)
 - $S_{\text{sum}}(l) = S_{\text{sum}}(n) = n$: 배열 전체가 임시기억장소에 복사
 - C : 배열의 첫번째 요소의 주소 전달
 - $S_{\text{sum}}(n) = 0$

```
float sum(float list[], int n)
{
    float tempsum = 0;
    int i;
    for (i=0; i<n; i++)
        tempsum += list[i];
    return tempsum;
}
```



공간 복잡도(3/3)

- [예제] rsum
 - 컴파일러가 매개 변수, 지역 변수, 매 순환 호출 시에 복귀 주소를 저장

```
float rsum(float list[], int n)
{
    int (n) return rsum(list, n-1) + list[n-1];
    return 0;
}
```

- 하나의 순환 호출을 위해 요구되는 공간
 - 두개의 매개 변수, 복귀 주소를 위한 바이트 수
 $= (\text{sizeof}(n)) + \text{sizeof}(\text{list}[] \text{의 주소}) + \text{sizeof}(\text{복귀주소}) = 12$
 - $N = \text{MAX_SIZE}$
 $S_{\text{rsum}}(\text{MAX_SIZE}) = 12 * \text{MAX_SIZE}$
 - 일반적으로 순환 함수는 반복 함수보다 큰 오버헤드를 가짐



시간 복잡도(1/7)

- 프로그램 P에 의해 소요되는 시간 : $T(P)$
= 컴파일 시간 + T_p (= 실행 시간)

컴파일 시간은 인스턴스 특성에 의존하지 않으므로 고정 공간 요구와 유사

- 예 : 수치 값을 더하고 빼는 간단한 프로그램

$$Tp(n) = C_a ADD(n) + C_s SUB(n) + C_l LDA(n) + C_{st} STA(n)$$

- C_a, C_s, C_l, C_{st} : 각 연산을 수행하기 위해 필요한 상수 시간

- $ADD, SUB, LDA, STA(n)$: 인스턴스 특성 n 에 대한 연산 실행 횟수

- 정의 : 프로그램 단계(program step)

– 실행 시간이 인스턴스 특성에 구문적으로 또는 의미적으로 독립성을 갖는 프로그램의 단위 $\Rightarrow 1 \text{ step!!}$

- $a = 2$

- $a = 2*b + 3*c/d - e + f/g/a/b/c$

※ 한단계 실행에 필요한 시간이 인스턴스 특성과 독립적이어야 함



시간 복잡도(2/7)

- 단계의 계산(방법 1)
 - 전역 변수 *count*의 사용
- 예제[리스트에 있는 수의 반복적 합산]

```
float sum(float list[], int n)
{
    float tempsum = 0; count++; /*for assignment */
    int i;
    for(i=0; i<n; i++) {
        count++;                /*for the for loop */
        tempsum += list[i]; count++; /*for assignment */
    }
    count++; /* last execution of for */
    count++; /* for return */ return tempsum;
}
```

```
float sum(float list[], int n)          /* 단순화된 프로그램 */
{
    float tempsum = 0;
    int i;
    for(i=0; i<n; i++)
        count += 2;
    count += 3;
    return 0;
}
```



시간 복잡도(3/7)

- 예제[리스트에 있는 수의 순환적 합산]

```
float sum(float list[], int n)
{
    count++;    /*for if conditional */
    if (n) {
        count++;    /*for return and rsum invocation */
        return rsum(list, n-1) + list[n-1];
    }
    count++;
    return list[0];
}
```

- $n = 0 \Rightarrow 2$ (if, 마지막 return)
- $n > 0 \Rightarrow 2$ (if, 처음 return): n 회 호출

$$\therefore 2n + 2 \text{ steps}$$

$$\text{※ } 2n + 3 \text{ (iterative)} > 2n + 2 \text{ (recursive)}$$

$\Rightarrow T_{\text{iterative}} > T_{\text{recursive}}$ (통상적으로 순환 함수가 작은 단계를 가지더라도 더 느리다)



시간 복잡도(4/7)

- 예제[행렬의 덧셈]

```
void add(int a[][MAX_SIZE], int b[][MAX_SIZE],
         int c[][MAX_SIZE], int rows, int cols)
{
    int i, j;
    for(i=0; i<rows; i++)
        for(j=0; j<cols; j++)
            c[i][j] = a[i][j] + b[i][j];
}
```

```
void add(int a[][MAX_SIZE], int b[][MAX_SIZE],
         int c[][MAX_SIZE], int rows, int cols)
{
    int i, j;
    for(i=0; i<rows; i++) {
        count++; /* for i for loop */
        for(j=0; j<cols; j++) {
            count++; /* for j for loop */
            c[i][j] = a[i][j] + b[i][j];
            count++; /* for assignment statement */
        }
        count++; /* last time of j for loop */
    }
    count++; /*last time of i for loop */
}
```

< count문이 첨가된 행렬의 덧셈 >



시간 복잡도(5/7)

```
void add(int a[][MAX_SIZE], int b[][MAX_SIZE],
         int c[][MAX_SIZE], int rows, int cols)
{
    int i, j;
    for(i=0; i<rows; i++)
        for(j=0; j<cols; j++) {
            count += 2;
            count+=2;
        }
    count++;
}
```

배열 크기 = $\text{rows} \times \text{cols}$

단계수 = $2\text{rows} \cdot \text{cols} + 2\text{rows} + 1$

$\text{rows} \gg \text{cols} \Rightarrow$ 행과 열을 교환



시간 복잡도(6/7)

- 단계의 계산(방법 2)
 - 테이블 방식(tabular method) : 단계수 테이블
 - 문장에 대한 단계수 : steps/execution, s/e
 - 문장이 수행되는 횟수 : 빈도수(frequency)
 - 비실행 문장의 빈도수 = 0
 - 총 단계수 : 빈도수 $\times s/e$
- [예제] 리스트에 있는 수를 합산하는 반복함수

문 장	s/e	빈도수	총단계수
<pre>float sum(float list[], int n) { float tempsum = 0; int i; for (i=0; i<n; i++) tempsum += list[i]; return tempsum; }</pre>	<pre>0 0 1 0 1 1 1 0</pre>	<pre>0 0 1 0 n+1 n 1 0</pre>	<pre>0 0 1 0 n+1 n 1 0</pre>
합 계			$2n+3$



시간 복잡도(7/7)

- [예제] 리스트에 있는 수를 합산하는 순환 함수

문 장	s/e 수	빈도수	총단계
<pre>float rsum(float list[], int n) { if (n) return rsum(list, n-1) + list[n-1]; return list[0]; }</pre>	$\begin{matrix} 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{matrix}$	$\begin{matrix} 0 \\ 0 \\ n+1 \\ n \\ 1 \\ 0 \end{matrix}$	$\begin{matrix} 0 \\ 0 \\ n+1 \\ n \\ 1 \\ 0 \end{matrix}$
합 계			$2n+2$

- [예제] 행렬의 덧셈

문 장	s/e 수	빈도수	총단계
<pre>void add(int a[][MAX_SIZE] ...) { int i, j; for (i=0, i<rows; i++) for (j=0, j<cols; j++) c[i][j] = a[i][j] + b[i][j]; }</pre>	$\begin{matrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 0 \end{matrix}$	$\begin{matrix} 0 \\ 0 \\ 0 \\ 0 \\ rows+1 \\ rows(cols+1) \\ rows \cdot cols \\ 0 \end{matrix}$	$\begin{matrix} 0 \\ 0 \\ 0 \\ 0 \\ rows+1 \\ rows \cdot cols + row \\ rows \cdot cols \\ 0 \end{matrix}$
합 계			$2rows \cdot cols + 2rows + 1$



점근 표기법 (O, Ω, Θ) (1/9)

- Worst case step count : 최대 수행 단계수
- Average step count : 평균 수행 단계수
 - ⇒ 동일 기능의 두 프로그램의 시간 복잡도 비교
 - 인스턴스 특성의 변화에 따른 실행 시간 증가 예측
- 정확한 단계의 계산 : 매우 어렵고 단계의 개념 자체가 부정확하여 무의미
- 근사치 사용: 단계 수의 차이가 클 때 유용하다
 - A의 단계수 = $3n + 3$, B의 단계수 = $100n + 10 \Rightarrow T_A \ll T_B$
 - $A \approx 3n$ (or $2n$), $B \approx 100n$ (or $80n, 85n$) $\Rightarrow T_A \ll T_B$
- (예제)
 - C_1 과 C_2 가 음이 아닌 상수일때
 - $C_1n^2 \leq T_p(n) \leq C_2n^2$ 또는 $T_Q(n,m) = C_1n + C_2m$
 - 예) $C_1=1, C_2=2, C_3=100$
 - $C_1n^2 + C_2n \leq C_3n, n \leq 98$
 - $C_1n^2 + C_2n > C_3n, n > 98$
 - 균형분기점(break even point) : $n = 98$



점근 표기법(2/9)

- 정의 [Big “oh”]
 - $n \geq n_0$ 인 모든 n 에 대해 $f(n) \leq cg(n)$ 인 조건을 만족하는 두 양의 상수 c 와 n_0 가 존재하기만 하면 $f(n) = O(g(n))$ 이다.
- 예제
 - $n \geq 2, 3n+2 \leq 4n \Rightarrow 3n+2 = O(n)$
 - $n \geq 3, 3n+3 \leq 4n \Rightarrow 3n+3 = O(n)$
 - $n \geq 10, 100n+6 \leq 101n \Rightarrow 100n+6 = O(n)$
 - $n \geq 5, 10n^2+4n+2 \leq 11n^2 \Rightarrow 10n^2+4n+2 = O(n^2)$
 - $n \geq 4, 6 \cdot 2^n + n^2 \leq 7 \cdot 2^n \Rightarrow 6 \cdot 2^n + n^2 = O(2^n)$
 - $n \geq 2, 3n+3 \leq 3n^2 \Rightarrow 3n+3 = O(n^2)$
 - $n \geq 2, 10n^2+4n+2 \leq 10n^4 \Rightarrow 10n^2+4n+2 = O(n^4)$
 - $n \geq n_0$ 인 모든 n 과 임의의 상수 c 에 대해 $3n+2 \leq c$ 가 false인 경우가 존재하면 $3n+2 \neq O(1), 10n^2+4n+2 \neq O(n)$
- ※ $O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n)$
- ※ $f(n) = O(g(n))$
 - $n \geq n_0$ 인 모든 n 에 대해 $g(n)$ 값은 $f(n)$ 의 상한값
 - $g(n)$ 은 조건을 만족하는 가장 작은 함수여야 함



점근 표기법(3/9)

- 정의 [Omega][$f(n) = \Omega g(n)$]
 - 모든 $n, n \geq n_0$ 에 대해서 $f(n) \geq cg(n)$ 을 만족하는 두 양의 상수 c 와 n_0 가 존재하기만 하면 $f(n) = \Omega(g(n))$ 이다.
- 예제
 - $n \geq 1, 3n+2 \geq 3n \Rightarrow 3n+2 = \Omega(n)$
 - $n \geq 1, 3n+3 \geq 3n \Rightarrow 3n+3 = \Omega(n)$
 - $n \geq 1, 100n+6 \geq 100n \Rightarrow 100n+6 = \Omega(n)$
 - $n \geq 1, 10n^2+4n+2 \geq n^2 \Rightarrow 10n^2+4n+2 = \Omega(n^2)$
 - $n \geq 1, 6 \cdot 2^n + n^2 \geq 2^n \Rightarrow 6 \cdot 2^n + n^2 = \Omega(2^n)$

※ $g(n)$: $f(n)$ 의 하한 값(가능한 큰 함수)



점근 표기법(4/9)

- 정의 [Theta][$f(n) = \Theta(g(n))$]
 - 모든 $n, n \geq n_0$ 에 대해서 $C_1g(n) \leq f(n) \leq C_2g(n)$ 을 만족하는 세 양의 상수 C_1, C_2 와 n_0 가 존재하기만 하면 $f(n) = \Theta(g(n))$ 이다.
- 예제
 - $n \geq 2, 3n \leq 3n+2 \leq 4n \Rightarrow 3n+2 = \Theta(n)$
 $c_1=3, c_2=4, n_0=2$
 - $3n+3 = \Theta(n)$
 - $10n^2+4n+2 = \Theta(n^2)$
 - $6 \cdot 2^n + n^2 = \Theta(2^n)$
 - $10 \cdot \log n + 4 = \Theta(\log n)$
 - ※ $g(n)$ 이 $f(n)$ 에 대해 상한 값과 하한 값을 모두 가지는 경우
 - ※ $g(n)$ 의 계수는 모두 1



점근 표기법(5/9)

- 예제
 - $T_{\text{sum}} = 2n + 3 \Rightarrow T_{\text{sum}}(n) = \Theta(n)$
 - $T_{\text{rsum}}(n) = 2n + 2 \Rightarrow \Theta(n)$
 - $T_{\text{add}}(\text{rows}, \text{cols}) = 2\text{row} \cdot \text{cols} + 2\text{rows} + 1 = \Theta(\text{rows} \cdot \text{cols})$
- 점근적 복잡도(asymptotic complexity: O , Ω , Θ)는 정확한 단계수의 계산 없이 쉽게 구함
- 예제[행렬 덧셈의 복잡도]

문 장	점근적 복잡도
<pre>void add(int a[][MAX_SIZE] ...) { int i, j; for (i=0, i<rows; i++) for (j=0, j<cols; j++) c[i][j] = a[i][j] + b[i][j] }</pre>	$ \begin{array}{c} 0 \\ 0 \\ 0 \\ \Theta(\text{rows}) \\ \Theta(\text{rows} \cdot \text{cols}) \\ \Theta(\text{rows} \cdot \text{cols}) \\ 0 \end{array} $
합 계	$\Theta(\text{rows} \cdot \text{cols})$



점근 표기법(6/9)

- [예제] 이진 탐색
 - 이진 탐색 함수(binsearch)에서 시간 복잡도 구하기
 - 인스턴스 특성 : 리스트 원소의 수 n
 - while 루프에서 반복시간 : $\log_2(n+1)$ 번
 - 매 반복 실행은 탐색되어야 할 list 세그먼트의 크기를 절반으로 감소. 최악의 경우 $\Theta(\log n)$ 번 반복
 - 한번 반복 실행하는데 $\Theta(1)$ 의 시간이 걸리기 때문에 binsearch의 최악의 경우 복잡도는 $\Theta(\log n)$
 - ※유의할 것은 while 루프의 첫번째 반복 실행에서 searchnum을 찾는 경우는 최상의 경우로서 복잡도가 $\Theta(1)$ 이 되는것임



점근 표기법 (7/9)

- [예제] 매직 스퀘어

15	8	1	24	17
16	14	7	5	23
22	20	13	6	4
3	21	19	12	10
9	2	25	18	11

< n=5인 매직 스퀘어 >

```
#include <stdio.h>
#define MAX_SIZE 15 /*maximum size of square */
Void main(void)
{ /* construct a magic square, iteratively */
    int square[MAX_SIZE][MAX_SIZE];
    int i, j, row, column;          /* indexes */
    int count;                      /* counter */
    int size;                       /* square size */

    printf("Enter the size of the square: ");
    scanf("%d", &size);
    /* check for input errors */
```



점근 표기법(8/9)

```
if (size < 1 || size > MAX_SIZE + 1) {
    fprintf(stderr, "Error! Size is out of range\n");
    exit(EXIT_FAILURE);
}
if (!(size % 2)) {
    fprintf(stderr, "Error! Size is even\n");
    exit(EXIT_FAILURE);
}
for (i=0; i<size; i++)
    for (j=0; j<size; j++)
        square[i][j] = 0;
square[0][(size-1) / 2] = 1; /* middle of first row */
/* i and j are current position */
i = 0;
j = (size-1) / 2;
for(count = 2; count <= size * size; count++) {
    row = (i-1 < 0) ? (size - 1) : (i-1); /*up*/
    column = (j-1 < 0) ? (size-1) : (j-1); /*left*/
```



점근 표기법(9/9)

```
    if(square[row][column])    /*down*/
        i = (++i) %size;
    else {                      /*square is unoccupied */
        i = row;
        j = (j-1<0) ? (size -1) : --j;
    }
    square[i][j] = count;
}
/* output the magic square */
printf("Magic Square of size %d : \n\n", size);
for (i=0; i<size; j++) {
    for (j=0; j<size; j++)
        printf("%5d", square[i][j]);
    printf("\n")
}
printf("\n\n");
}
```

< magic square 프로그램 >



실용적인 복잡도

- 함수값

$\log n$	n	$n \log n$	n^2	n^3	2_n
0	1	0	1	1	2
1	2	2	4	8	4
2	4	8	16	64	16
3	8	24	64	512	256
4	16	64	256	4,096	65,536
5	32	160	1024	32,768	4,294,967,296

- 함수 2^n 은 n 이 증가함에 따라 더욱 빠르게 증가함
- 프로그램 실행을 위해 2^n 단계가 필요하다면 $n=40$ 일 때
요구되는 단계 수는 대략 1.1×10^{12}
- 1초에 10개의 단계를 수행하는 컴퓨터라면 약 18.3분 요구
- $n=50$ 이면 13일, $n=60$ 이면 310.56년, $n=100$ 이면 4×10^{13} 년
따라서 유용도는 n 이 아주 작을 때($n \leq 40$)로 제한됨



성능 측정(1/6)

- 시간 측정
 - 방법 1: clock을 사용
 - 방법 2: time을 사용

	방법 1	방법 2
시작시간	<code>start = clock();</code>	<code>start = time(NULL);</code>
종료시간	<code>stop = clock();</code>	<code>stop = time(NULL)</code>
반환타입	<code>clock_t</code>	<code>time_t</code>
초 단위 결과	<code>Duration = ((double)(stop-start))/CLOCKS_PER_SEC</code>	<code>duration = (double) difftime(stop, start);</code>

< C 에서 시간을 재는 사건 >



성능 측정(2/6)

- [예제] 선택 함수의 최악의 성능

```
#include <stdio.h>
#include <time.h>
#include "selectionSort.h"
#define MAX_SIZE 1001
void main(void)
{
    int i, n, step = 10;
    int a[MAX_SIZE];
    double duration;
    clock_t start;

    /* time for n = 0, 10, ..., 100, 200, ..., 1000 */
    printf("    n    time\n");
    for(n=0; n<= 1000; n += step)

        /* get time for size n */
```



성능 측정(3/6)

```
/* initialize with worst-case data */
for (i=0; i<n; i++)
    a[i] = n-i;

start = clock();
sort(a, n);
duration = ((double)) (clock() - start))
           / CLOCK_PER_SEC;
printf("%6d  %f\n", n, duration);
if (n == 100) step = 100;
}
}
```

< 선택 정렬 함수의 첫번째 시간 측정 프로그램 >



성능 측정(4/6)

```
#include <stdio.h>
#include <time.h>
#include "selectionSort.h"
#define MAX_SIZE 1001
void main(void)
{
    int i, n, step = 10;
    int a[MAX_SIZE];
    double duration;

    /* time for n = 0, 10, ..., 100, 200, ..., 1000 */
    printf("    n    time\n");
    for(n=0; n<= 1000; n += step)

        /* get time for size n */
        long repetitions = 0;
        clock_t start = clock( );
        do
        {
```



성능 측정 (5/6)

```
{
    repetitions++;

    /* initialize with worst-case data */
    for (i=0; i<n; i++)
        a[i] = n - i;

    sort(a, n);
} while (clock( ) - start < 1000);
    /* repeat until enough time has elapsed */

duration = ((double) (clock() - start))
            / CLOCK_PER_SEC;
duration /= repetitions;
printf(“%6d %9d %f\n”), n, repetitions, duration);
if (n == 100) step = 100;
}
} < 보다 정확한 선택 정렬 함수의 시간 측정 프로그램 >
```



성능 측정(6/6)

n	repetitions	time	n	repetitions	time
0	8690714	0.000000	100	44058	0.000023
10	2370915	0.000000	200	12585	0.000079
20	604948	0.000002	300	5780	0.000173
30	329505	0.000003	400	3344	0.000299
40	205605	0.000005	500	2096	0.000477
50	145353	0.000007	600	1516	0.000660
60	110206	0.000009	700	1106	0.000904
70	85037	0.000012	800	852	0.001174
80	65751	0.000015	900	681	0.001468
90	54012	0.000019	1000	550	0.001818

< 선택 정렬의 최악의 경우에서의 성능(단위는 초) >



Homework I

- Read Chapter 1, Chapter 2
- 연습문제 : 1, 3, 4, 6

